1. Problem statement and our understanding

Problem (Vanguard India, Community Stewardship): "*Empowering the Next Generation Through Financial Literacy.*" The audience is Grade 11–12 students (16–18 years) in Hyderabad, reached through NGO partners (pilot of ~1,500 students), across five pillars: getting banked and digital payments, planning and budgeting, saving and borrowing, understanding investing, and staying safe from fraud.

What we think the real problem is:

- **Money lessons don't stick when they are lectures.** Students forget rules like "keep an emergency fund" or "never share your PIN" because they never feel what happens when they break them. With real money, consequences arrive weeks later and mistakes cost too much to learn from.
- **The setting is hard:** low-end, often shared phones; patchy or no internet in classrooms; several languages (English, Telugu, Hindi, Urdu); and NGO facilitators who need to see where a class is stuck.
- **Guardrails:** students are minors, so we must not collect real financial or family data. We must **educate, not recommend** investments (no real funds, stocks, banks or apps). Any projection must be shown as an assumption with a range, not a promise.

2. Idea / abstract

PaisaPath is a simulator, not a course. "*Learn money by living it.*" A student builds a fictional **character** through 10 picture-based questions (who they are, where their money comes from, their goal, their money personality). They then **live that character's financial life month by month for a simulated year**. The life starts at the very beginning, opening a bank account, and moves through stages in order:

S0 Get Banked → **S1** Digital Money (UPI) → **S2** First Money & Budget → **S3** Save & Track → **S4** Big Decisions & Borrowing → **S5** Grow Money → **S6** Protect & Look Ahead

A **Stay Safe thread** drops simulated scam events (fake KYC SMS, UPI collect requests, loan apps, "double your money" tips) into every stage. Students can get scammed, overspend or miss a goal **safely**. They can then **rewind** to a key decision, choose differently, and compare **Parallel Lives** side by side under the *same luck*, so they see that the decision made the difference, not chance.

Mode	What it does
LifePath (main)	The 12-month persona life (S0–S6). Each stage ends with a short mastery gate.
Rewind & Parallel Lives	Branch from checkpoints and compare timelines, with a rule-based explanation and a "Future Me" projection to age 25.
Arcade	Focused mini-games, e.g. Bank Counter, Jar Split budgeting, Debt Cost Ladder, Investing ABC lessons, Time Machine (500 futures), Stock Market and Mutual Fund simulators, UPI Simulator, Scam Inbox, The Call.
Quick Play	5–10 minute scenarios and a 2-minute Daily Challenge with spaced review of weak concepts.
Facilitator dashboard	Class code and QR, stage unlocks, a concept heatmap, top misconceptions with discussion prompts, stuck students.

Why it fits the brief: it is persona-led and relatable (local festivals, kirana, Irani chai), game-based, works offline on shared phones, supports several languages and audio, and is NGO-ready. Scaling comes through **content as data**: a teacher can add a scenario as JSON without a developer.

3. High-level approach and architecture

UI layer (React + TypeScript + Tailwind, PWA): onboarding, character builder, LifePath screens, Rewind viewer, Arcade games, facilitator dashboard. *No game logic in components.*	**Core** (pure TypeScript): persona classifier, simulation engine `reduce(state, action) → state' + events`, stage scheduler, **seeded RNG + event tape**, branch manager, fictional-asset market simulator with a Monte-Carlo runner, mastery model.	**Content packs** (static JSON): scenarios, life events, lessons, concepts and misconceptions, string bundles (EN/TE/HI/UR), audio. Checked by a schema validator.
Local storage (IndexedDB via Dexie): profiles with a picture-PIN (hashed), saves, timeline snapshots, an event outbox. **Fully playable offline.**	**Service worker**: caches the app shell and content, and sends the outbox in the background when online. It never blocks play.	**Cloud** (Supabase: Postgres, row-level security, edge functions): classes, pseudonymous players, gameplay events, aggregates for facilitators.

Pipeline: install via class QR → set up (language, accessibility, class code, avatar, picture-PIN; *no email or phone*) → 10-question character builder → persona + starting finances → pre-check → monthly LifePath loop → stage gates → rewind / Arcade / Quick Play → year-end review, Future Me and post-check → anonymous events sync → facilitator heatmap

Key design decisions

- **Deterministic engine.** The same seed and the same choices always give the same state. This makes rewind, fair "same luck" comparisons, offline play and unit testing possible. Shocks, festivals, scams and market moves come from a seeded *event tape* that stays the same across branches.
- **One learning loop for every activity:** Hook → Play → Result → Why? (misconception detection) → Concept → Try again → Check → Reward. Play uses eight reusable **verbs**: Choose, Allocate, Commit, Triage, Rank, Predict, Judge, Act in-situ.
- **Mastery, not score.** Each concept has a mastery value from 0 to 1, updated by `m ← m + rate·(target − m)`, with red/amber/green status, hints when a student is stuck, and spaced review.
- **Simulated-only and privacy-first.** We never touch the student's real money, messages or family finances. Investing screens use only fictional assets, show ranges and an assumptions drawer, and carry the banner "*Simulation based on assumptions. Not a prediction or advice.*" There is no AI in the gameplay path; all explanations are rule-based and testable.

Built so far: project scaffold with CI (typecheck, lint, tests, build); the student setup flow, player picker and picture-PIN; the facilitator dashboard on Supabase with 7 RLS-protected tables; the offline storage and event outbox; the seeded market engine, financial math library and 500-futures runner; and the first interactive Investing ABC lessons with the Predict verb. **143 unit tests pass.** Next: the life engine and LifePath stages S0–S6, Rewind, the Everyday Money and Stay Safe games, and the remaining investing games.

4. Planned testing methodology

Level	What we test and how
Unit tests (Vitest, run in CI on every PR)	All core logic is pure functions, so it is tested directly: the engine reducer, financial math (compounding, inflation, EMI), the market simulator, mastery updates and lesson logic. The persona classifier is tested **exhaustively over every answer combination** to check that each one produces a valid persona and starting state.
Determinism and replay	Same seed + same choices → identical serialized state in every game. Rewind tests check that tape events land in the same month in both timelines and that the original timeline is never changed.
Content validation	A schema validator checks every scenario for missing strings, concepts or verbs in all languages. A **word-list test** fails the build if a real brand, bank, fund or company name appears in content.
Simulation sanity checks	Monte-Carlo outputs are checked against closed-form expectations (mean growth, spread widening over time, crash recovery). Performance budget: 500 futures in under 1 s on a low-end phone (currently about 30 ms for 20 years × 500 futures on a laptop).
Device and accessibility	Every screen at 360×640, with large text and in Urdu right-to-left; airplane-mode play → reconnect → the facilitator heatmap updates; first load under 3 MB; a test on a real low-end Android phone.
Learning effectiveness	A pre-check and a post-check drawn from **separate item pools on the same concepts** measure learning gain on unseen items. We also track the misconception rates the "Why?" step detects. During the hackathon we will run quick hallway playtests with peers acting as 16–18 year olds and time each module (target: one LifePath month in 5–8 minutes).